

ANALYSIS AND DESIGN OF ALGORITHMS

Instructor: Dr. Mert Büyükdede

Selection Sort Algorithm

Selection Sort is a simple and intuitive sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and placing it in the correct position.

It is one of the sorting algorithms developed to keep data in order in memory.

It is simply found where the smallest number in the array is at each step.

With this number, the number at the beginning of the array is replaced and the smallest numbers are selected and put at the beginning.



Selection Sort Algorithm

We can think of these data as the determinant fields of a composition. For example, such as a citizenship number or a student number.

Therefore, it can be used in the case of sorting students according to their numbers.

The operation of the selective sorting is shown step by step in the NEXT part.



Steps of Selection Sort:

1. Start with the first element and assume it is the smallest.
2. Compare it with the remaining elements to find the actual smallest element.
3. Swap the smallest element found with the first element.
4. Move to the next element and repeat the process for the rest of the list.
5. Continue this process until the entire list is sorted.

Let's sort the array below using the selection sort method.

[29, 10, 14, 37, 13]

Find the smallest element (10) and swap it with the first element:

[10, 29, 14, 37, 13]

Find the smallest element in the remaining part (13) and swap it with the second element:

[10, 13, 14, 37, 29]

The next smallest element is already in place (14).

Find the smallest in the remaining part (29) and swap with 37:

[10, 13, 14, 29, 37]

The array is now sorted.

Selection Sort Algorithm

Step 0: Set the starting step as $i=0$.

Step 1: Find the place of the smallest number after the i th row in the array. The smallest number in this array is 1 and is in the 5th row.

Step 2: Replace the number in the i th row of the array with this smallest number:
1,7,2,9,6,5,3,7

Step 3: Increase i by one and go to step 1.

Therefore, after step 3, $i=1$ and then the first number 1 is skipped and the smallest number among the remaining numbers, 7,2,9,6,5,3,7, is found. This number is 2 and its order is 2. The next number is replaced with the number in its place and the result is: 1,2,7,9,6,5,3,7.

Let's try to pseudo code for selection sort

```
SelectionSort(A)
  for i ← 0 to length(A) - 2
    minIndex ← i
    for j ← i + 1 to length(A) - 1
      if A[j] < A[minIndex]
        minIndex ← j
    if minIndex ≠ i
      swap A[i] and A[minIndex]
```



Explanation:

A: The array to be sorted

minIndex: The index of the smallest value found in the unsorted part

i: The current position in the array

j: The index used to search for the smallest value

Example Java Code

Let's try to write java code for selection sort

```
package selectionsort;

import java.util.Arrays;

public class SelectionSort {

    public static void sort(int[] arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[minIndex]) minIndex = j;
            }
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }

    public static void main(String[] args) {
        int[] numbers = {29, 10, 14, 37, 13};
        System.out.println("Unsorted: " + Arrays.toString(numbers));
        sort(numbers);
        System.out.println("Sorted: " + Arrays.toString(numbers));
    }
}
```

In the example code, two loops are written inside each other, the smallest number is found in the inner loop, and this process is repeated each time in the outer loop.

The complexity of this algorithm is $O(n^2)$ because n numbers are processed in each step and this process is repeated n times.

✓ Advantages of Selection Sort:

1. Simplicity:

1. The algorithm is easy to understand and implement, making it a good choice for teaching basic sorting concepts.

2. Works Well on Small Datasets:

1. For small lists, Selection Sort can be sufficient since its overhead is low compared to more complex sorting algorithms.

3. In-Place Sorting Algorithm:

1. Selection Sort does not require extra memory beyond a few variables, as it sorts the list in place. This makes it **space-efficient** with an $O(1)$ space complexity.

4. Performs a Fixed Number of Swaps:

1. Unlike Bubble Sort, which swaps elements multiple times, Selection Sort minimizes swaps by only swapping once per iteration. This can be beneficial in scenarios where writing to memory is costly.

5. Independent of Initial Order:

1. Unlike some sorting algorithms, the performance of Selection Sort does not degrade if the array is already sorted or nearly sorted. The number of comparisons remains the same in all cases.

✗ Disadvantages of Selection Sort:

1. Poor Time Complexity for Large Datasets:

The algorithm always requires $O(n^2)$ time, even if the list is already sorted. This makes it impractical for large datasets compared to algorithms like Merge Sort ($O(n \log n)$) or QuickSort ($O(n \log n)$ on average).

2. Not a Stable Sorting Algorithm:

Selection Sort does not preserve the relative order of equal elements. If there are duplicate values, their order might change, which can be problematic in certain applications like sorting databases.

3. Inefficient Compared to Other Sorting Algorithms:

Even though it minimizes swaps, the number of comparisons remains high ($O(n^2)$) making it much slower than optimized algorithms like QuickSort, Merge Sort, or even Insertion Sort in some cases.

4. Bad Performance for Nearly Sorted Data:

Unlike Insertion Sort, which can perform efficiently on nearly sorted data ($O(n)$ in the best case), Selection Sort does not take advantage of pre-existing order and still runs in $O(n^2)$.

5. Limited Practical Use:

Due to its inefficiency on large datasets, Selection Sort is rarely used in real-world applications. It is mostly employed in educational contexts or in cases where memory constraints are extreme, and stability is not required.

Insertion Sort Algorithm

Insertion Sort is a simple and efficient sorting algorithm that works similarly to the way people arrange playing cards in their hands. It builds the sorted list one element at a time by taking each element and inserting it into its correct position.

Insertion sort is a sorting algorithm that is relatively simple to program, but its performance is relatively slow compared to sorts such as merge sort and quick sort.



Steps of Insertion Sort:

1. Assume the first element is already sorted.
2. Pick the next element and compare it with the elements in the sorted part.
3. Shift larger elements to the right to make space for the current element.
4. Insert the current element into its correct position.
5. Repeat this process for all elements until the entire list is sorted.

The numbers to be sorted are:

33 44 21 83 56 73 22

We start sorting these numbers from the first number

in the first pass, only the number 33 is sorted (i.e. we do nothing):

33| 44 21 83 56 73 22 (the | sign shows the numbers we have sorted up to that moment. We accept the numbers to the left of this sign "|" as sorted. And we take the number to the right of it in each pass)

Insertion Sort Algorithm

the number we will sort **in the second pass** is 44.

We compare 33 and 44, 33 is small so they do not change places:

33 44 | 21 83 56 73 22

in the third pass the next number is 21. 21 is compared with 44 and since 21 is smaller they are replaced with 44:

33 21 44 | 83 56 73 22 (our pass is not over yet because 21 is smaller than 33:)

21 33 44 | 83 56 73 22 (now we have completed the 3rd pass and we can get the next number 83:)



Insertion Sort Algorithm

in the fourth pass there is 83:

21 33 44 83 | 56 73 22 (this pass is over quickly because 83 is bigger than 44 and just seeing this is enough for us to stop, after all we have sorted up to 56)

In the fifth pass we will sort 56:

21 33 44 56 83 | 73 22 (here 56 and 83 are compared and 56 is moved to the left of 83. This is because 56 is smaller than 83. This pass ends here because 56 is larger than 44)

In the sixth pass it is 73's turn

21 33 44 56 73 83 | 22 (our sort ends in one step again because 73 is smaller than 83 and larger than 56)

Insertion Sort Algorithm

In the seventh pass we will insert the number 22 and step by step we slide 22 in the array until we see a number that is less than 22:

21 33 44 56 73 22 83 |

21 33 44 56 22 73 83 |

21 33 44 22 56 73 83 |

21 33 22 44 56 73 83 |

21 22 33 44 56 73 83 |

Insertion Sort Algorithm

The performance of insertion sort is $O(n^2)$.

This is because it requires as many passes as the number of elements in the array, and in the worst case, it requires as many shifts as the number of elements in each pass.

So the worst case of insertion sort is a reverse-sorted list. Considering the example above:

A sequence of order 83 73 56 44 33 22 21 is the longest-sorted sequence. In contrast, given a sorted sequence, $2n$ accesses to the sequence are sufficient.

Example : Let's create algorithm for Insertion Sort

```
InsertionSort(A)
  for i ← 1 to length(A) - 1
    key ← A[i]
    j ← i - 1
    while j ≥ 0 and A[j] > key
      A[j + 1] ← A[j]
      j ← j - 1
    A[j + 1] ← key
```

 Explanation:

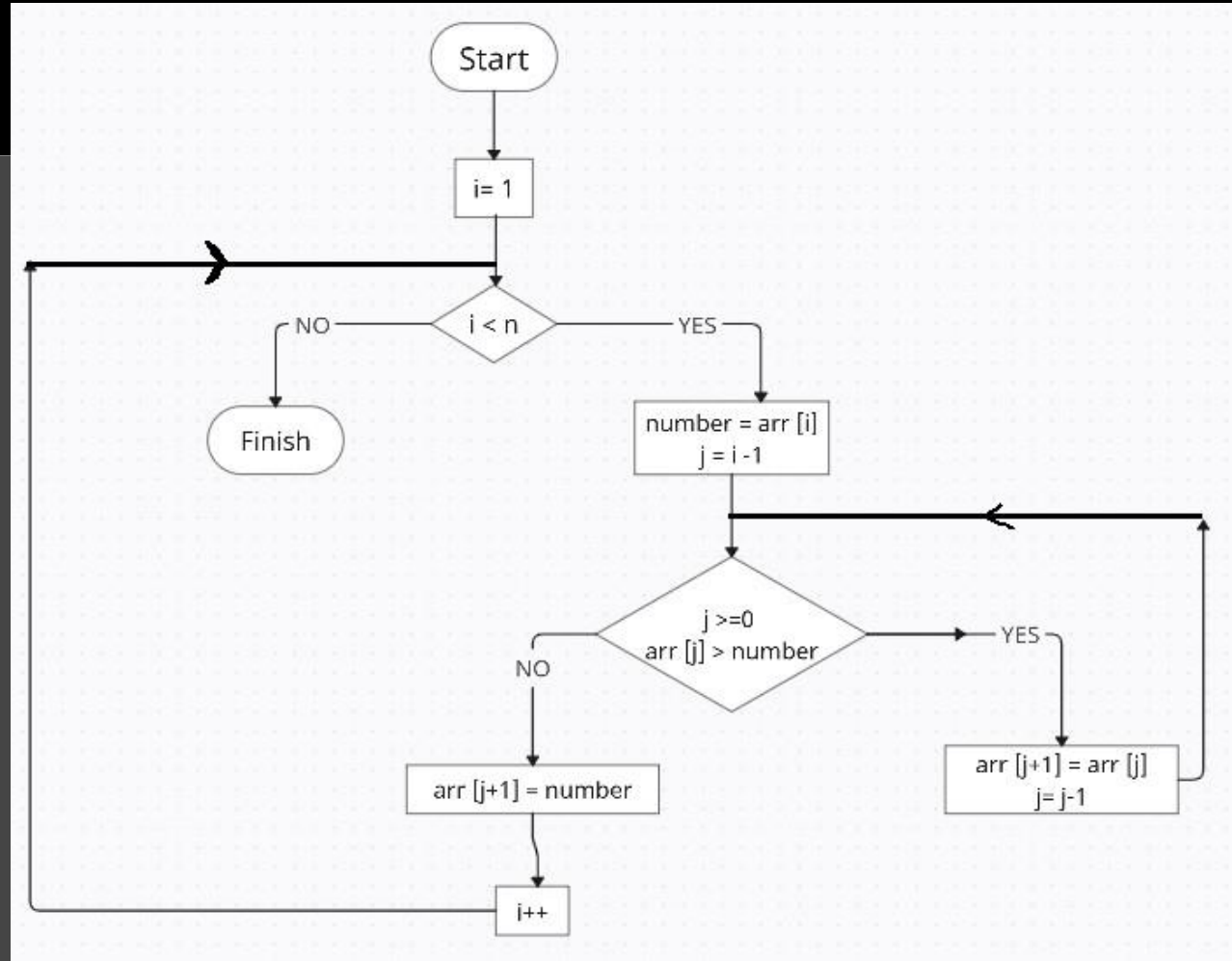
A: The array to be sorted

key: The current value to be inserted into the sorted part

j: The index that moves leftward to find the correct position for key

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

Example : Let's create a diagram for Insertion Sort



Example : Let's create java code for Insertion Sort (Data Set: 2, 5, 12, 3, 9, 4, 17, 6)

```
package insertionsort;

import java.util.Arrays;

public class InsertionSort {

    public static void sort(int[] arr) {
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            // Elemanı doğru konumuna yerleştirmek için kaydırma işlemi
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        int[] numbers = {11, 16, 8, 7, 20, 4, 14};

        System.out.println("*** Insertion Sort ***");
        sort(numbers);
        System.out.println("Sorted Array: " + Arrays.toString(numbers));
    }
}
```

Advantages & Disadvantages of Insertion Sort

✓ Advantages:

1. Efficient for Small and Nearly Sorted Data:

If the list is nearly sorted, Insertion Sort performs in $O(n)$ time, which is much better than Selection Sort.

2. Stable Sorting Algorithm:

It preserves the relative order of equal elements, making it useful for sorting records.

3. In-Place Sorting:

Does not require extra memory beyond a few variables ($O(1)$ space complexity).

4. Adaptive Nature:

Performs well on small datasets and almost-sorted data, making it preferable in such cases.

✗ Disadvantages:

1. Inefficient for Large Datasets:

It has an $O(n^2)$ worst-case time complexity, making it impractical for large arrays.

2. More Shifting Operations:

Unlike Selection Sort, which minimizes swaps, Insertion Sort shifts multiple elements when inserting a value.

3. Slower than Advanced Sorting Algorithms:

Merge Sort and QuickSort (both $O(n \log n)$) are much faster for large datasets.

When to Use Insertion Sort?

- ✓ When sorting small datasets.
- ✓ When the list is already or nearly sorted.
- ✓ When stability (maintaining order of equal elements) is important.

Activity: Insertion Sort vs. Selection Sort - Which One is More Efficient?

Goal 1: Students complete the empty methods in the code below.

```
public class SortingAlgorithms {  
    // Selection Sort Method  
    public static void selectionSort(int[] arr) {  
        // Implement Selection Sort here  
    }  
  
    // Insertion Sort Method  
    public static void insertionSort(int[] arr) {  
        // Implement Insertion Sort here  
    }  
}
```

HOMEWORK

Goal 2: Measure and Compare Performance

Measure execution time and compare the efficiency of each algorithm.

Data set 1

{64, 25, 12, 22, 11, 90, 3, 75}

Data set 2

The array consist of 20 elements

Data set 3

The array consist of 1000 elements

HOMEWORK

Goal 3: Real-World Scenario - Best Algorithm for Each Case

Scenario:

A logistics company needs to sort incoming orders for delivery scheduling. The company presents three different datasets:

- 1 Completely random order numbers
- 2 Nearly sorted orders (with minor changes)
- 3 A very large, unsorted order list

Task:

Determine which algorithm is best for each case and explain why.